Chapter 4 Programming Assignment: Parallel Heat Diffusion with MPI

The challenge

Write a Python program using **mpi4py** and **NumPy** to simulate heat moving along a one-dimensional rod. Split the rod across MPI ranks, exchange boundary values with nonblocking messages, and stop when the whole rod has converged or a maximum number of steps has been reached.

This assignment builds on Chapter 4's collectives, uneven `Scatterv` partitions, nonblocking halo exchange, global reductions, and performance analysis. Your final numerical answer must come from the distributed calculation, not from a serial calculation on rank 0.

Learning outcomes

By completing the assignment, you should be able to:

1. Compute balanced counts and displacements for an arbitrary number of cells and ranks.
2. Distribute and collect `float64` arrays with `Scatterv` and `Gatherv`.
3. Exchange halo values with `Irecv`, `Isend`, and `Waitall` without misusing active buffers.
4. Use `allreduce` to make a consistent stopping decision on every rank.
5. Validate a distributed numerical result against an independent serial implementation.
6. Measure parallel runtime and explain the effects of communication and load balance.

Numerical problem

The rod has `N` cells indexed `0` through `N-1`. Initially, every cell is zero except the middle cell:

```
u[N // 2] = 1.0
```

At each step, update every cell simultaneously using the previous step's values:

```
new[i] = old[i] + lam * (left[i] - 2 * old[i] + right[i])
```

For a cell at either physical end of the rod, the value beyond the rod is always `0.0` (a fixed-zero boundary). Use `0 < lam <= 0.5`. Do **not** update a cell in place while another cell still needs its old value.

After each step, compute the largest absolute change on each rank. The global change is the maximum across **all** ranks. Stop early when `tol > 0` and the global change is less than `tol`; otherwise stop after `max_steps`. When `tol = 0`,

run exactly `max_steps` steps. Report the number of steps actually completed and whether the tolerance was reached.

Hand-check example

With `N=5`, `lam=0.25`, and one step, the initial rod and the correct updated rod are:

```
initial: [0.00, 0.00, 1.00, 0.00, 0.00]
step 1:  [0.00, 0.25, 0.50, 0.25, 0.00]
global maximum change: 0.50
```

Your result must be the same regardless of the number of MPI ranks.

Required implementation

Create `heat_mpi.py` with the following command-line interface:

```
mpiexec -n 4 python -m mpi4py heat_mpi.py --cells 17 --steps 100 --lam 0.25 --tol 1e-6
```

The program must meet **all** of these requirements:

1. Rank 0 parses and validates the arguments. Broadcast either a valid configuration or an error state so all ranks exit consistently. Require `N >= 1`, `max_steps >= 1`, `0 < lam <= 0.5`, and `tol >= 0`.
2. Use balanced contiguous blocks. For `N=17, P=4`, the counts must be `[5, 4, 4, 4]` and the displacements `[0, 5, 9, 13]`. Displacements are measured in `float64` elements, not bytes.
3. Rank 0 creates the initial `float64` rod. Distribute it with **`comm.Scatterv`**. Allocate each rank's receive array to its exact local count.
4. Store two ghost cells around each nonempty local block. An empty rank owns no cells and posts no halo messages. It still joins every collective used by the other ranks.
5. At every step, post receives before sends. Use **`Irecv` and `Isend`** for the left and right halo values, with consistent source, destination, and tag directions. Use `MPI.PROC_NULL` for missing physical neighbors. Make sure a physical-boundary ghost remains `0.0` on every step, including after swapping arrays.
6. While the requests are active, update only cells whose required neighbors are already local. Call **`MPI.Request.Waitall`** before reading a received ghost or changing a send buffer. Then update the local boundary cells. Handle a block of one cell without updating it twice.
7. Compute the local maximum absolute difference between old and new owned cells. Use **`comm.allreduce(..., op=MPI.MAX)`** to obtain the global change. All ranks must call this collective in the same order and make the same stopping decision.
8. After the final step, collect the owned values on rank 0 with **`comm.Gatherv`**. Do not gather the whole rod inside the time-stepping loop.

9. Implement a separate serial reference function on rank 0. Compare the final distributed vector with the serial result using `np.allclose` with `rtol=1e-10` and `atol=1e-12`. The serial function must follow the same stopping rule, so also compare the number of completed steps.
10. Print one concise final result line from rank 0 containing `N`, `P`, `steps_done`, `global_change`, `converged`, and `PASS` or `FAIL`. A failed comparison must produce a nonzero program exit status on all ranks.

You may adapt ideas from `examples/chapter4/nonblocking_halo.py` and `examples/chapter4/dot_product_scatterv.py`, but your submitted program must perform the complete simulation. Use Python throughout; no C or C++ implementation is required.

Empty partitions

Your code must work when `N < P`. Balanced partitioning puts one cell on each of ranks `0` through `N-1`; higher ranks have zero cells. An active rank's neighbors are the adjacent **active** ranks. Empty ranks must not send or receive halo data, but they must take part in `allreduce` and the final `Gatherv`. For example, `N=3, P=4` leaves rank 3 empty.

Required test cases

Run the following commands and record the final result line for each. All must report `PASS`.

Case	Command	What it checks
A	`mpiexec -n 1 python -m mpi4py heat_mpi.py --cells 5 --steps 1 --lam 0.25 --tol 0`	Hand-check result and physical boundaries
B	`mpiexec -n 4 python -m mpi4py heat_mpi.py --cells 17 --steps 40 --lam 0.25 --tol 0`	Uneven counts and displacements
C	`mpiexec -n 4 python -m mpi4py heat_mpi.py --cells 3 --steps 20 --lam 0.25 --tol 0`	An empty rank

Case	Command	What it checks
D	`mpiexec -n 4 python -m mpi4py heat_mpi.py --cells 1 --steps 10 --lam 0.25 --tol 0`	One active rank with one cell
E	`mpiexec -n 4 python -m mpi4py heat_mpi.py --cells 23 --steps 500 --lam 0.25 --tol 1e-5`	Global early stopping

For case A, also print or record the full final vector and show that it matches `[0, 0.25, 0.5, 0.25, 0]`. Include one deliberate invalid-input test and show that it terminates cleanly without leaving other ranks waiting in a collective.

Performance investigation

Use `N=100003`, `max_steps=200`, `lam=0.25`, and `tol=0` for a fixed-work comparison. If your machine cannot run this size comfortably, state the largest size you used and use the **same** size for every process count.

1. Measure the time for the distributed stepping loop with `MPI.Wtime()`. Exclude argument parsing, initial distribution, final gathering, validation, and printing. Synchronize once immediately before timing. At the end, report the **maximum** elapsed time across ranks, since the slowest rank determines completion.
2. Run with `P=1`, `P=2`, and `P=4`, if your machine supports all three. Repeat each configuration at least three times and report the median.
3. Create a table and a graph of runtime against process count. Calculate speedup `S(P)=T(1)/T(P)` and efficiency `E(P)=S(P)/P`.
4. In 150–250 words, interpret the results. Discuss halo message size, global `allreduce`, array size per rank, and whether the measured results agree with your expectations. A slower parallel run is a valid result if it is measured and explained honestly.

Do not label the Chapter 4 latency–bandwidth model as a measured runtime. State your CPU, operating system, MPI implementation, Python version, and process-placement method if known.

Submission

Submit a single ZIP archive containing:

- `heat_mpi.py` — the executable Python source;
- `README.md` — setup instructions and exact run commands;

- `report.pdf` — at most five pages, with a short algorithm explanation, a rank/halo diagram for `N=17, P=4`, the required test-result table, the performance table and graph, and the performance discussion.

Your report should identify which arrays are active during nonblocking communication and explain why the interior update is safe before `Waitall`. Include the counts and displacements for `N=17, P=4` and `N=3, P=4`.

Assessment rubric (100 marks)

Criterion	Marks
Correct balanced counts, displacements, `Scatterv`, and `Gatherv`	20
Correct nonblocking halos, tags, boundaries, and buffer lifetime	25
Simultaneous stencil update, global convergence, and consistent rank control flow	20
Serial validation and all required edge-case tests	15
Performance measurements, table, graph, and interpretation	15
Clear, reproducible source and README	5

Stretch challenge (optional, up to 10 bonus marks)

Implement a second communication mode that uses blocking `Sendrecv` for the halo exchange. Compare its fixed-step runtime with the nonblocking mode for the same inputs. Explain when the nonblocking version has independent work available and why its use does not by itself guarantee a speedup. Keep both modes numerically equivalent.